TECHNICKÁ UNIVERZITA V KOŠICIACH
FAKULTA ELEKTROTECHNIKY A INFORMATIKY

Klasifikácia medicínskych dát pomocou algoritmu XGBoost

Dokumentácia projektu

2026 **Bc. Svitlana Chystiakova**

1. Cieľ práce a rozsah riešenia

Práca sa zameriava na binárnu klasifikáciu medicínskych dát z datasetu *Breast Cancer Wisconsin*. Hlavným cieľom bolo vytvoriť kompletný postup spracovania dát zahŕňajúci:

- načítanie dát,
- prieskum datasetu (EDA),
- rozdelenie dát na trénovaciu a testovaciu množinu,
- trénovanie modelu XGBoost,
- ladenie hyperparametrov,
- vyhodnotenie výsledkov,
- export modelu pre použitie v aplikácii.

Súčasťou riešenia je aj implementácia vlastného zjednodušeného modelu vychádzajúceho z princípu XGBoost, ktorá predstavuje jednu z hlavných častí zadania. Cieľom tejto implementácie je demonštrovať fungovanie algoritmu, najmä využitie gradientov, hessiánov a iteratívneho zlepšovania modelu.

2. Teoretický základ algoritmu XGBoost

XGBoost (Extreme Gradient Boosting) je pokročilý algoritmus strojového učenia založený na princípe **gradient boosting**, ktorý patrí medzi ensemble metódy.

Jeho hlavnou myšlienkou je kombinovať viacero slabších modelov (rozhodovacích stromov) do jedného silného modelu.

- **Princíp gradient boostingu**

Gradient boosting funguje tak, že model sa vytvára postupne v iteráciách. V každom kroku sa nový model učí na základe chýb predchádzajúceho modelu.

Celkový model po m krokoch môžeme zapísať ako:

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$$

kde:

- $F_{m-1}(x)$ je aktuálny model,
- $h_m(x)$ je nový strom,
- η je learning rate.

Learning rate riadi, aký veľký vplyv má každý nový strom na výsledný model. Nižšie hodnoty vedú k pomalšiemu, ale stabilnejšiemu učeniu a znižujú riziko preučenia, avšak vyžadujú väčší počet stromov.

Optimalizačná úloha

Cieľom XGBoost je minimalizovať nasledujúcu funkciu:

$$L = \Sigma l(y_i, \hat{y}_i) + \Sigma \Omega(f_k)$$

kde:

- $l(y_i, \hat{y}_i)$ je loss funkcia (napr. log loss) , ktorá meria chybu modelu,
- $\Omega(f_k)$ je regularizačný člen, ktorý penalizuje zložitosť jednotlivých stromov.

- **Regularizácia**

Na rozdiel od klasického gradient boostingu XGBoost obsahuje regularizáciu, ktorá penalizuje príliš komplexné modely.

$$\Omega(f) = \gamma T + 1/2\lambda \Sigma w_j^2$$

kde:

- T je počet listov stromu,
- w_j sú váhy listov,
- γ určuje penalizáciu za počet listov (zložitosť stromu),
- λ sú riadi veľkosť váh listov.

Regularizácia pomáha zabrániť preučeniu modelu tým, že uprednostňuje jednoduchšie a stabilnejšie riešenia.

- **Taylorov rozvoj**

XGBoost aproximuje loss funkciu pomocou Taylorovho rozvoja druhého rádu:

$$L \approx \Sigma[g_i \cdot f(x_i) + 1/2h_i \cdot f(x_i)^2] + \Omega(f)$$

kde:

- g_i je gradient (prvá derivácia funkcie straty),
- h_i je hessián (druhá derivácia funkcie straty),
- f_i je výstup stromu pre daný príklad.

Táto aproximácia umožňuje efektívne optimalizovať model a zohľadniť nielen veľkosť chyby, ale aj jej zmenu.

- **Gradienty a hessiány**

Gradient a hessián sa počítajú ako:

$$g_i = \partial l(y_i, \hat{y}_i)/\partial \hat{y}_i$$
$$h_i = \partial^2 l(y_i, \hat{y}_i)/\partial \hat{y}_i^2$$

Gradient určuje smer, ktorým sa má model zlepšiť, zatiaľ čo hessián poskytuje informáciu o zakrivení funkcie straty.

- **Výpočet hodnoty listu**

Pre každý list stromu sa vypočíta optimálna hodnota:

$$w *= -\Sigma g_i/(\Sigma h_i + \lambda)$$

Gradient určuje smer, ktorým sa má model zlepšiť, zatiaľ čo hessián poskytuje informáciu o zakrivení funkcie straty.

- **Výpočet zisku (split gain)**

Pri rozhodovaní o rozdelení uzla sa používa tzv. gain:

$$Gain = 1/2\left[\left(G_L^2/(H_L + \lambda)\right) + \left(G_R^2/(H_R + \lambda)\right) - \left(G^2/(H + \lambda)\right)\right] - \gamma$$

kde:

- G, H sú sumy gradientov a hessiánov.

Tento vzorec určuje, či rozdelenie uzla zlepší model. Rozdelenie sa vykoná iba vtedy, ak je hodnota zisku kladná.

3. Dataset a prieskum dát

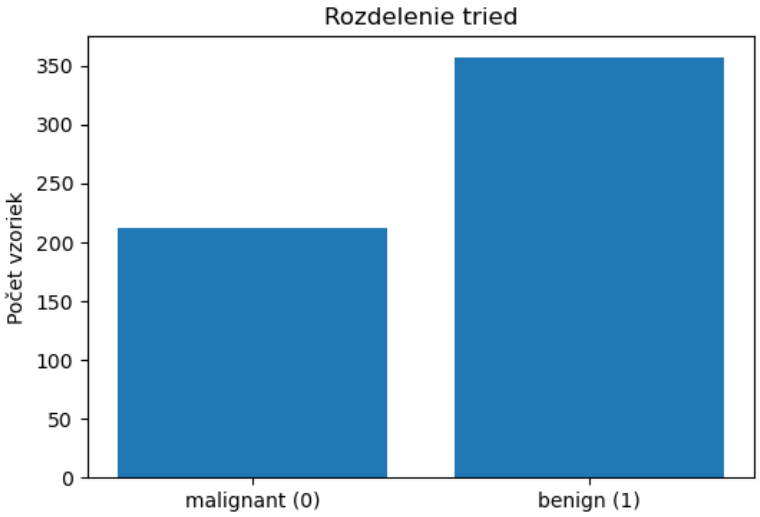
Použitý dataset obsahuje 569 vzoriek a 30 numerických príznakov. Cieľová premenná je binárna:

- 0 = malignant
- 1 = benign

V datasete je 357 benígnych a 212 malígnych vzoriek, čo znamená miernu nevyváženosť tried v prospech triedy benign.

- Počet riadkov: 569
- Počet príznakov: 30
- Triedy: malignant (0), benign (1)
- Chýbajúce hodnoty: neobsahuje chýbajúce hodnoty

Kontrola dátových typov v notebooku potvrdzuje, že všetky atribúty sú numerické typu float64. To zjednodušuje predspracovanie, pretože nebolo potrebné kódovanie kategórií ani imputácia chýbajúcich hodnôt. A zároveň XGBoost nevyžaduje škálovanie dát, preto nebolo potrebné vykonávať normalizáciu.



Obr. 1 Zastúpenie tried malignant a benign v celej množine

4. Hodnotenie modelu a použité metriky

Na vyhodnotenie klasifikačného modelu sa používa tzv. matica zámen (confusion matrix), ktorá prehľadne zobrazuje výsledky klasifikácie pre jednotlivé triedy.

Táto matica pozostáva zo štyroch základných prvkov:

- **TP (True Positive)** – počet prípadov, keď model správne identifikoval pozitívnu triedu (napr. chorý pacient bol správne klasifikovaný ako chorý)
- **TN (True Negative)** – počet prípadov, keď model správne identifikoval negatívnu triedu (napr. zdravý pacient bol správne klasifikovaný ako zdravý)
- **FP (False Positive)** – počet prípadov, keď model nesprávne označil negatívnu triedu ako pozitívnu (napr. zdravý pacient bol klasifikovaný ako chorý)
- **FN (False Negative)** – počet prípadov, keď model nesprávne označil pozitívnu triedu ako negatívnu (napr. chorý pacient bol klasifikovaný ako zdravý)

V notebooku bola implementovaná funkcia *evaluate_model()*, ktorá:

- počíta základné klasifikačné metriky,
- generuje classification report,
- zobrazuje konfúznu maticu.

Na hodnotenie modelu boli použité nasledovné metriky:

Metrika	Výpočet / význam	Interpretácia
Accuracy	$ACC = \frac{(TP + TN)}{(TP + TN + FP + FN)}$	Udáva celkový podiel správnych predikcií.
Precision	$Precision = \frac{TP}{(TP + FP)}$	Vyjadruje, aký podiel pozitívnych predikcií bol správny. Je dôležitá v situáciách, kde chceme minimalizovať počet falošne pozitívnych výsledkov.
Recall	$Recall = \frac{TP}{(TP + FN)}$	Vyjadruje, akú časť skutočne pozitívnych prípadov model zachytil. V medicínskom kontexte je táto metrika kľúčová, pretože minimalizuje počet falošne negatívnych prípadov, teda situácií, keď model neidentifikuje chorého pacienta.
F1-score	$F1 = \frac{2 \cdot (Precision \cdot Recall)}{(Precision + Recall)}$	Predstavuje harmonický priemer precision a recall. Používa sa najmä v prípade, keď je potrebné nájsť rovnováhu medzi týmito dvoma metrikami.
ROC-AUC	plocha pod ROC krivkou	Hodnotí schopnosť modelu odlišovať triedy pri rôznych prahoch. Vyššia

		hodnota ROC-AUC znamená lepšiu diskriminačnú schopnosť modelu.

Pre komplexné zhodnotenie modelu je preto vhodné kombinovať viaceré metriky, nie sa spoliehať iba na accuracy. Z hľadiska medicínskej aplikácie je najdôležitejší recall, keďže minimalizuje riziko nesprávneho klasifikovania chorého pacienta ako zdravého.

5. Implementácia XGBoost z knižnice

Model bol implementovaný pomocou knižnice XGBoost, ktorá predstavuje optimalizovanú a široko používanú implementáciu gradient boosting algoritmu.

Na ladenie hyperparametrov bol využitý nástroj GridSearchCV, ktorý umožňuje systematicky testovať rôzne kombinácie parametrov a vybrať konfiguráciu s najlepšími výsledkami.

5.1. Mriežka hyperparametrov a ich význam

Pri ladení modelu boli testované nasledovné hyperparametre:

Hyperparameter	Skúšané hodnoty	Význam
n_estimators	[50, 100, 150]	Určuje počet stromov v modeli. Vyšší počet stromov zvyšuje schopnosť modelu zachytiť komplexné vzťahy v dátach, avšak zároveň môže viesť k preučeniu a predlžuje čas trénovania.
max_depth	[3, 4, 5]	Určuje maximálnu hĺbku jednotlivých stromov. Väčšia hĺbka umožňuje modelu lepšie zachytiť nelineárne vzťahy, no zvyšuje riziko overfittingu.
learning_rate	[0.05, 0.1, 0.2]	Riadi veľkosť príspevku každého stromu do výsledného modelu. Nižšie hodnoty vedú k stabilnejšiemu učeniu a lepšej generalizácii, avšak vyžadujú viac iterácií.
subsample	[0.8, 1.0]	Určuje podiel trénovacích vzoriek použitých pri tvorbe každého stromu. Hodnoty menšie ako 1.0 zavádzajú náhodnosť a pomáhajú redukovať overfitting.
colsample_bytree	[0.8, 1.0]	Určuje podiel príznakov použitých pri stavbe každého stromu. Znižuje koreláciu medzi stromami a zlepšuje robustnosť modelu.

Najlepšia kombinácia parametrov získaná pomocou GridSearchCV bola**:**

colsample_bytree	learning_rate	max_depth	n_estimators	subsample
0.8	0.1	4	100	1.0

Optimalizačné kritérium bolo nastavené na recall (scoring = 'recall') s využitím 5-násobnej krížovej validácie. Takýto prístup bol zvolený zámerne, keďže v medicínskych úlohách je prioritou minimalizácia falošne negatívnych prípadov, teda situácií, keď model neidentifikuje chorého pacienta.

5.2. Výsledky knižničného modelu

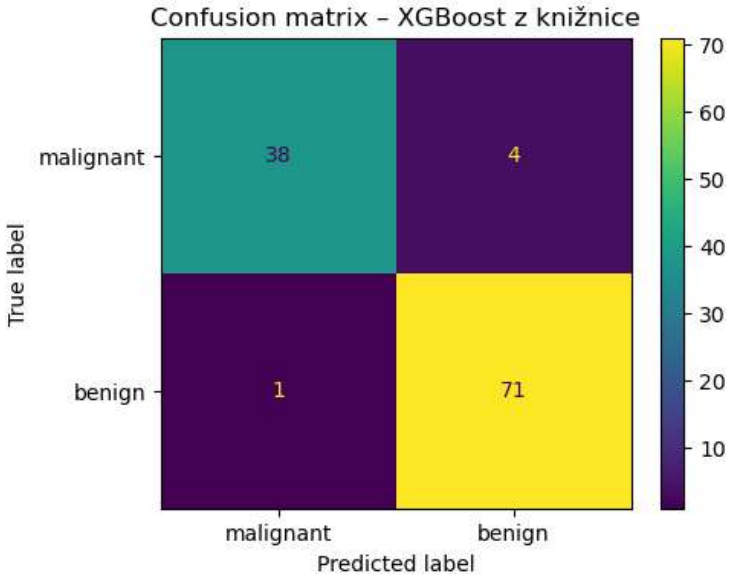
Model dosiahol na testovacej množine nasledovné výsledky:

Metrika	Hodnota
Accuracy	0.9561
Precision	0.9467
Recall	0.9861
F1-score	0.9660
ROC-AUC	0.9964

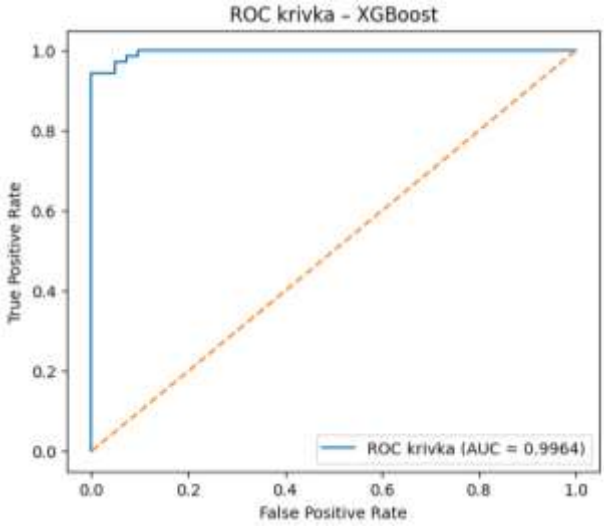
Výsledky poukazujú na veľmi vysokú kvalitu modelu. Celková presnosť presahuje 95 %, pričom hodnota recall dosahuje až 98.6 %, čo je z pohľadu medicínskej aplikácie mimoriadne dôležité.

Konfúzna matica ukazuje, že model správne identifikoval väčšinu prípadov oboch tried.

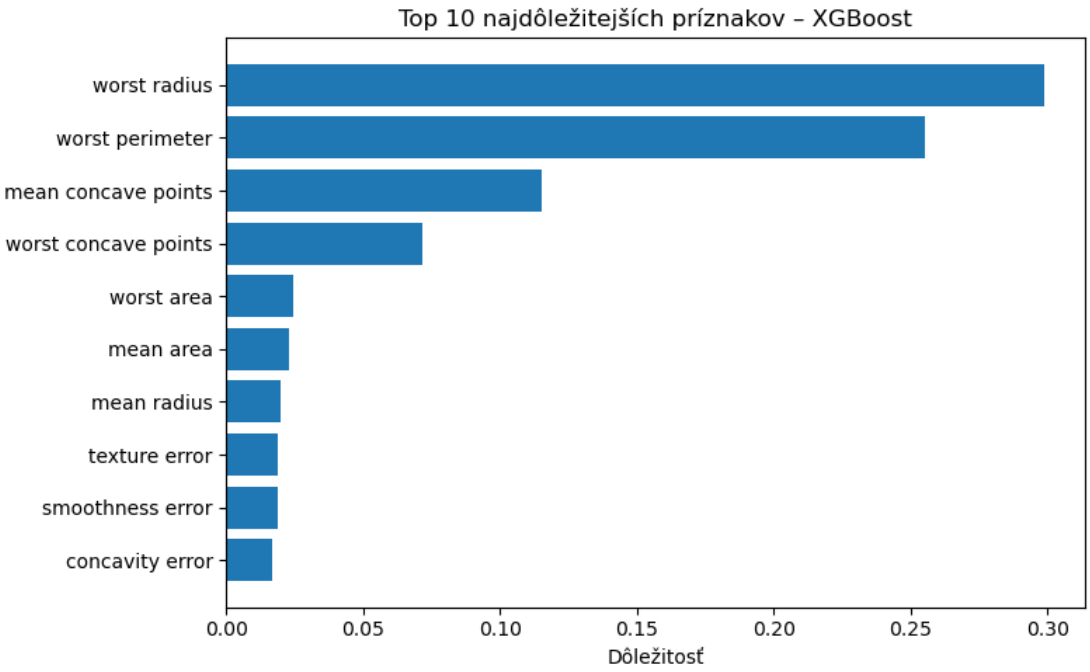
Konkrétne:



Obr. 2 Konfúzna matica modelu XGBoost z knižnice



Obr. 3 ROC krivka modelu XGBoost



Obr. 4 Top 10 najdôležitejších príznakov podľa modelu XGBoost

Analýza feature importance ukazuje, že najvýznamnejšie príznaky boli - *worst radius, worst perimeter*, *mean concave points, worst concave points, worst area*.

Tieto atribúty opisujú najmä veľkosť a tvar analyzovaných štruktúr, čo je v súlade s medicínskym kontextom úlohy, kde práve tieto charakteristiky zohrávajú kľúčovú úlohu pri rozlišovaní medzi benígnymi a malígnymi prípadmi.

6. Vlastná zjednodušená implementácia princípu XGBoost

Okrem knižničnej implementácie bola vytvorená aj vlastná zjednodušená verzia algoritmu XGBoost.

Cieľom tejto časti nebolo reprodukovať kompletný algoritmus, ale detailne pochopiť jeho mechanizmus – najmä spôsob, akým sa model postupne zlepšuje pomocou gradientov a hessiánov.

Navrhnutá implementácia pozostáva z troch hlavných komponentov:

- sigmoid funkcie,
- triedy **XGBoostStump**, ktorá reprezentuje jednoduchý rozhodovací strom s jedným splitom,
- triedy **SimpleXGBClassifier**, ktorá riadi celý boosting proces.

6.1. Prehľad kódu

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

class XGBoostStump:
    def __init__(self, lambd=1.0):
        self.lambd = lambd
        self.feature_index = None
        self.threshold = None
        self.left_weight = None
        self.right_weight = None

    def fit(self, X, g, h):
        ...
        gain = 0.5 * (
            (G_L ** 2) / (H_L + self.lambd) +
            (G_R ** 2) / (H_R + self.lambd) -
            ((G_L + G_R) ** 2) / (H_L + H_R + self.lambd)
        )
        ...
    def predict(self, X):
        ...
class SimpleXGBClassifier:
    def __init__(self, n_estimators=30, learning_rate=0.1, lambd=1.0):
        self.n_estimators = n_estimators
        self.learning_rate = learning_rate
        self.lambd = lambd

    def fit(self, X, y):
        p_mean = np.clip(np.mean(y), 1e-6, 1 - 1e-6)
        self.base_score = np.log(p_mean / (1 - p_mean))
        ...
```

```
    gradients = prob - y
    hessians = prob * (1 - prob)
    ...
    F += self.learning_rate * update

def predict_proba(self, X):
    ...

def predict(self, X):
    ...
```

6.2. Vysvetlenie jednotlivých funkcií

Funkcia / metóda	Úloha v algoritme
sigmoid(z)	Táto funkcia prevádza lineárne skóre modelu na pravdepodobnosť v intervale ⟨0,1⟩. Je nevyhnutná pre logistickú klasifikáciu, pretože výstup modelu je najprv vo forme log-odds, ktoré je potrebné transformovať na interpretovateľnú pravdepodobnosť príslušnosti k triede.
XGBoostStump.__init__()	Inicializuje regularizačný parameter λ a premenné, do ktorých sa budú ukladať parametre najlepšieho nájdeného splitu (index príznaku, prahová hodnota a váhy listov).
XGBoostStump.fit(X, g, h)	Pre každý príznak a každú možnú prahovú hodnotu skúša rozdelenie dát a vypočítava hodnotu gain. Na základe tohto kritéria vyberie split, ktorý najviac znižuje chybu modelu, a následne vypočíta optimálne váhy pre ľavý a pravý list.
XGBoostStump.predict(X)	Na základe uloženého splitu rozhodne, či daná vzorka patrí do ľavej alebo pravej vetvy stromu, a vráti príslušnú hodnotu listu.
SimpleXGBClassifier.__init__()	Nastavuje základné hyperparametre modelu, konkrétne počet boosting iterácií, learning rate a regularizačný parameter λ.
SimpleXGBClassifier.fit(X, y)	Inicializuje model pomocou log-odds odhadu a následne v každej iterácii: - vypočíta pravdepodobnosti, - určí gradienty a hessiány, - natrénuje nový stump, - aktualizuje skóre modelu F. Týmto spôsobom sa model postupne zlepšuje a minimalizuje chybu.
SimpleXGBClassifier.predict_proba(X)	Vypočíta výsledné skóre ako súčet základného odhadu a príspevkov všetkých stumpov, ktoré následne prevedie

	pomocou sigmoid funkcie na pravdepodobnosti jednotlivých tried.
SimpleXGBClassifier.predict(X)	Na základe vypočítanej pravdepodobnosti použije rozhodovací prah a priradí každej vzorke konečnú triedu.

6.3. Hyperparametre vlastnej implementácie

Vo vlastnej implementácii boli použité parametre n_estimators = 40, learning_rate = 0.2 a lambd = 1.0.

Parameter	Vysvetlenie
n_estimators = 40	Určuje počet boosting iterácií, teda počet postupne pridávaných stumpov. Každý nový stump sa učí na základe chýb aktuálneho modelu a postupne zlepšuje jeho predikciu.
learning_rate = 0.2	Riadi veľkosť aktualizácie modelu v každej iterácii. Určuje, aký veľký vplyv má výstup nového stumpu na výsledné skóre modelu. Vyššie hodnoty vedú k rýchlejšiemu učeniu, avšak môžu znižovať stabilitu modelu.
lambd = 1.0	Predstavuje regularizačný parameter použitý pri výpočte gain a váh listov. Pomáha obmedziť príliš veľké hodnoty aktualizácií a tým znižuje riziko preučenia modelu.

6.4. Prečo nejde o plnú implementáciu XGBoost

Napriek tomu, že vlastná implementácia dosahuje veľmi dobré výsledky, nejde o kompletnú implementáciu algoritmu XGBoost.

Navrhnuté riešenie bolo zámerne zjednodušené s cieľom zvýšiť prehľadnosť kódu a umožniť lepšie pochopenie základných princípov gradient boostingu.

V porovnaní s knižničnou implementáciou chýbajú najmä tieto vlastnosti:

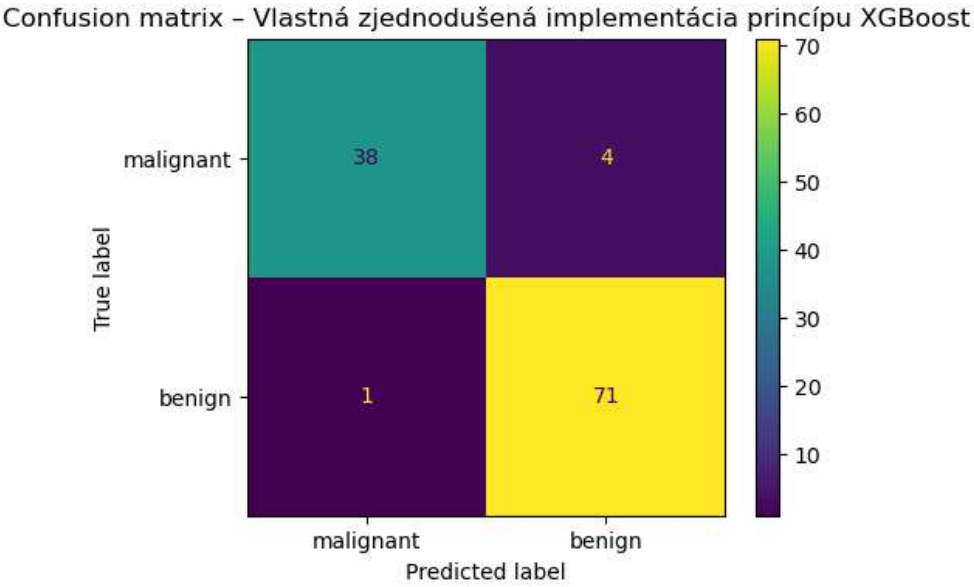
- Používajú sa iba jednoduché rozhodovacie stromy (stumpy), teda stromy s jedným splitom, namiesto hlbších a komplexnejších stromov.
- Nie sú implementované pokročilé stratégie vyhľadávania splitov ani optimalizačné techniky, ktoré výrazne zvyšujú efektivitu knižničnej verzie.
- Chýba pruning stromov, ako aj mechanizmy pre efektívnu prácu so sparse dátami a chýbajúcimi hodnotami.
- Regularizácia je zjednodušená iba na parameter λ vo výpočte váh listov, pričom plná implementácia obsahuje viacero regularizačných prvkov.
- Implementácia nevyužíva paralelizáciu výpočtov ani histogram-based algoritmy, ktoré sú kľúčové pre spracovanie veľkých datasetov

Z tohto dôvodu je vhodné túto časť práce označiť ako zjednodušenú implementáciu princípu XGBoost, nie ako kompletnú reprodukciu knižničného riešenia.

6.5. Výsledky vlastnej implementácie

Metrika	Hodnota
Accuracy	0.9561
Precision	0.9467
Recall	0.9861
F1-score	0.9660
ROC-AUC	0.9927

Zjednodušený model dosiahol na testovacej množine takmer rovnaké metriky ako knižničná verzia, hoci ROC-AUC je o niečo nižšie. To potvrdzuje, že implementácia zachytáva základný princíp boostingu pomerne dobre.



Obr. 5 Konfúzna matica modelu XGBoost

7. Porovnanie oboch prístupov

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
XGBoost knižnica	0.9561	0.9467	0.9861	0.9660	0.9964
Zjednodušený princíp XGBoost	0.9561	0.9467	0.9861	0.9660	0.9927

Z tabuľky je zrejmé, že oba modely dosahujú takmer identické hodnoty základných metrík, ako sú accuracy, precision, recall a F1-score.

To naznačuje, že aj zjednodušená implementácia dokáže veľmi dobre zachytiť rozhodovacie hranice v dátach a poskytovať presné predikcie.

Rozdiel sa prejavuje najmä v metrike ROC-AUC, kde knižničná verzia dosahuje mierne lepší výsledok. Tento rozdiel poukazuje na lepšiu schopnosť modelu rozlišovať medzi triedami pri rôznych rozhodovacích prahoch.

Z hľadiska medicínskej aplikácie je obzvlášť dôležitá vysoká hodnota recall, ktorá je v oboch prípadoch veľmi vysoká (0.9861). To znamená, že model dokáže spoľahlivo identifikovať pozitívne prípady a minimalizovať počet falošne negatívnych predikcií.

8. Export modelu a Shiny aplikácia

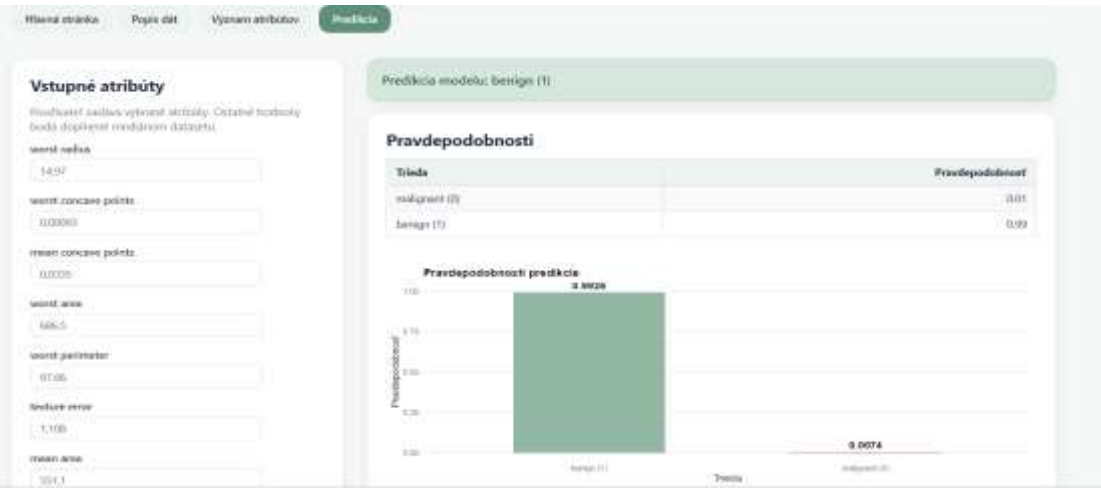
Na záver boli pripravené exportné súbory, aby bolo možné model načítať v Shiny aplikácii bez nového trénovania. Notebook ukladá model XGBoost vo formáte JSON a zároveň vytvára pomocné CSV súbory s datasetom, feature importance, mediánmi príznakov, názvami príznakov a základnými informáciami o modeli.

Na základe exportovaných dát bola vytvorená jednoduchá interaktívna aplikácia pomocou frameworku Shiny, ktorá umožňuje vykonávať predikciu v reálnom čase na základe zadaných vstupných atribútov.

Používateľ môže zadať vybrané hodnoty príznakov a poskytne výslednú klasifikáciu spolu s pravdepodobnosťami jednotlivých tried.

Hlavným prínosom aplikácie je možnosť praktického využitia modelu bez potreby práce s kódom, čo umožňuje jednoduchú interpretáciu výsledkov a demonštráciu fungovania modelu.

Obr. 6 Hlavná obrazovka Shiny aplikácie s implementovaným modelom XGBoost



Obr. 7 Výsledok predikcie v Shiny aplikácii pre zadané vstupné atribúty

9. Záver

V tejto práci bol navrhnutý a vyhodnotený klasifikačný model založený na algoritme XGBoost.

Najskôr bol vykonaný prieskum dát, následne bol natrénovaný model pomocou knižničnej implementácie s ladením hyperparametrov a bola vytvorená aj vlastná zjednodušená verzia algoritmu.

Výsledky ukázali, že:

- knižničná implementácia XGBoost dosahuje veľmi vysokú presnosť a výborné hodnoty všetkých metrík,
- vlastná implementácia dosahuje takmer identické výsledky v základných metrikách (accuracy, precision, recall a F1-score),
- rozdiel medzi modelmi sa prejavuje najmä v metrike ROC-AUC, čo poukazuje na mierne lepšiu diskriminačnú schopnosť knižničného modelu,
- boostingový prístup umožňuje postupné zlepšovanie modelu na základe chýb predchádzajúcich iterácií.

Na základe dosiahnutých výsledkov možno konštatovať, že algoritmus XGBoost je veľmi vhodný na klasifikáciu tabuľkových dát.

Zároveň sa ukázalo, že aj zjednodušená implementácia dokáže dosiahnuť porovnateľné výsledky, čo potvrdzuje robustnosť gradient boosting prístupu a jeho schopnosť efektívne modelovať komplexné vzťahy v dátach.